

# 30 Days SQL Micro Course

## Day 2 – Types of Commands in SQL(DDL)

### CREATE Command (DDL)

The CREATE command is used to create database objects such as databases, tables, views, indexes, etc.

---

#### 1. CREATE DATABASE

Syntax

```
CREATE DATABASE database_name;
```

Example

```
CREATE DATABASE CollegeDB;
```

This creates a new database named CollegeDB.

---

#### 2. CREATE TABLE

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

This creates a table named Students with three columns.

---

#### 3. CREATE VIEW

**A view is a virtual table based on a query.**

**Syntax**

```
CREATE VIEW view_name AS  
SELECT column1, column2  
FROM table_name;
```

**Example**

```
CREATE VIEW StudentNames AS  
SELECT StudentID, Name  
FROM Students;
```

**This creates a view containing only the StudentID and Name columns.**

---

#### **4. CREATE INDEX**

**An index improves search performance.**

**Syntax**

```
CREATE INDEX index_name  
ON table_name (column_name);
```

**Example**

```
CREATE INDEX idx_name  
ON Students (Name);
```

**This creates an index on the Name column.**

---

#### **5. CREATE TABLE with Constraints**

**Syntax**

```
CREATE TABLE table_name (  
    column_name datatype CONSTRAINT  
);
```

**Example**

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18)  
);
```

**Here:**

- **PRIMARY KEY** → Unique identifier
- **NOT NULL** → Cannot be empty
- **CHECK** → Age must be 18 or above

## **How to Create a Database in PostgreSQL**

**Step 1: Open pgAdmin**

- Open pgAdmin 4
- Enter your PostgreSQL password if asked.

---

**Step 2: Connect to PostgreSQL Server**

**In the left panel:**

**Servers**

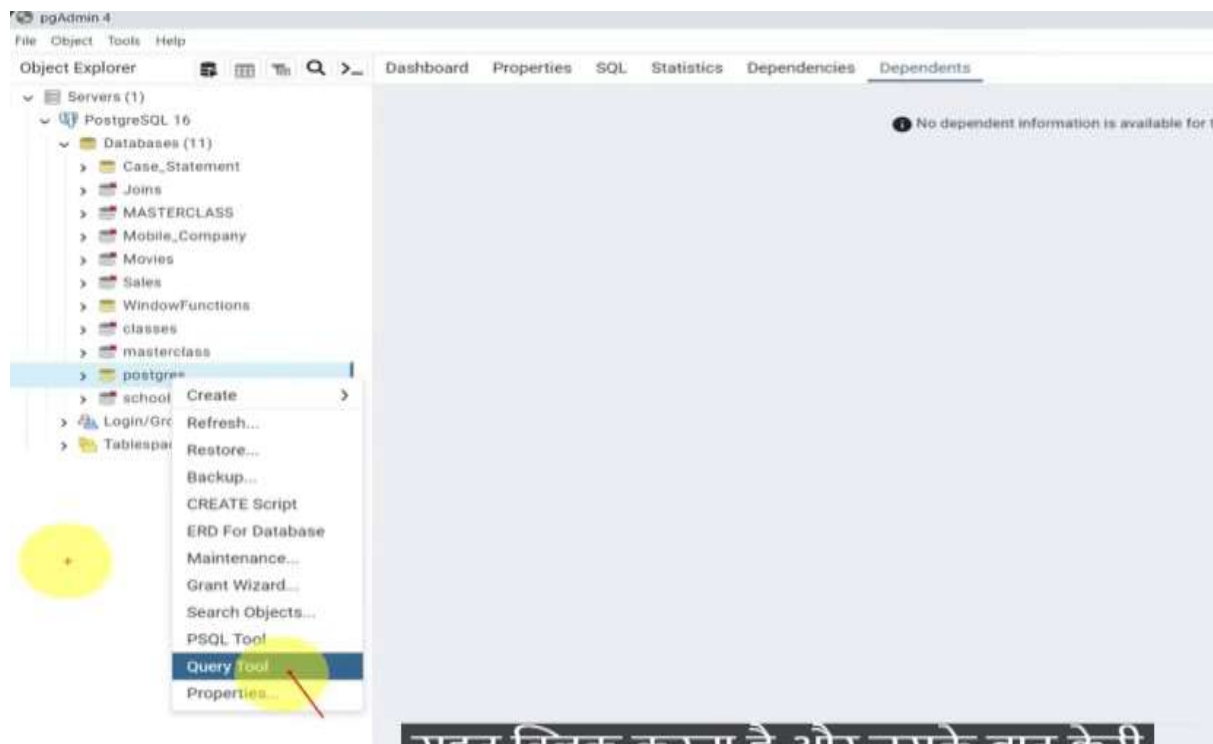
└ PostgreSQL

**Click on it to connect.**

---

**Step 3: Create Database**

1. **Expand Servers**
2. **Expand PostgreSQL**
3. **Right-click Databases**
4. **Right Click postgres**
5. **Click Create → Database**



---

#### Step 4: Enter Database Details

Example:

Database Name: company\_db

Owner: postgres

Click Save.

---

#### Step 5: Verify

You should now see:

Databases

└─ postgres

└─ company\_db

---

#### Create Database Using SQL

Open Query Tool and run:

**CREATE DATABASE company\_db;**

To check:

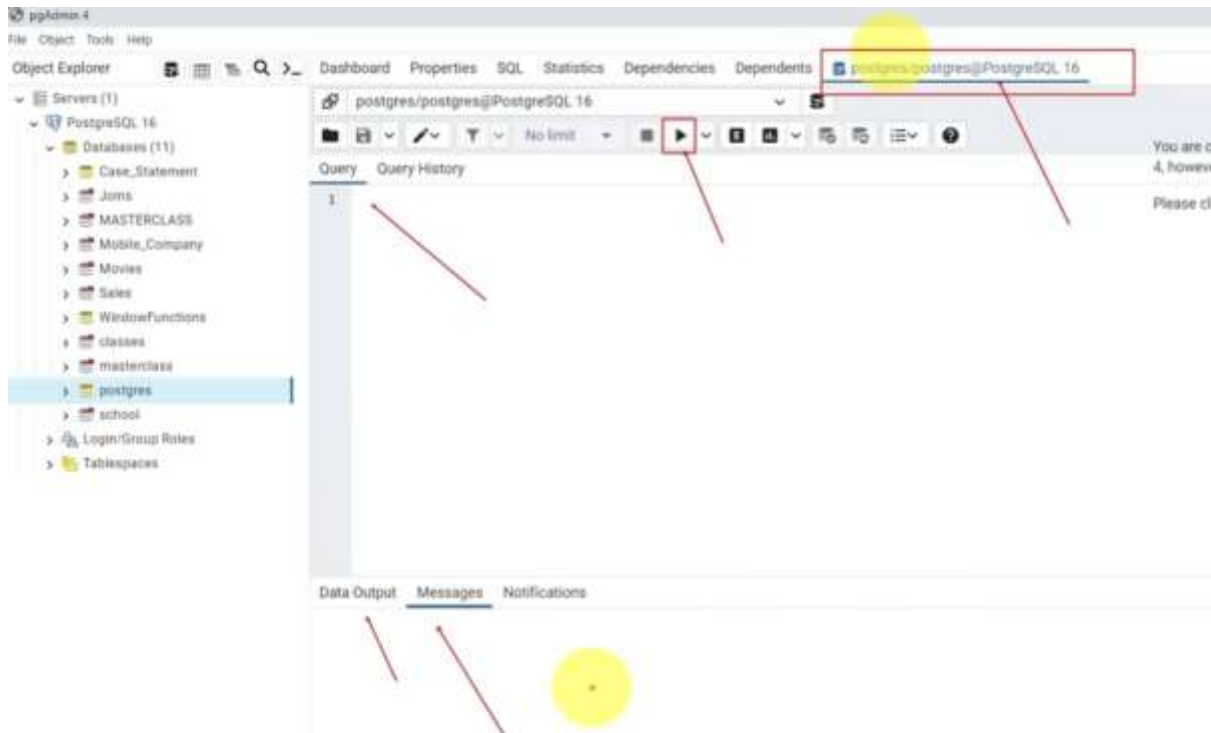
```
SELECT datname
FROM pg_database;
```

Where,

**SELECT = Show**

**datname = Database Name**

**FROM pg\_database = From the PostgreSQL database list**



## Create a Table in PostgreSQL (Step by Step)

### Step 1: Open pgAdmin

- Open pgAdmin 4
- Enter your PostgreSQL password if asked.

---

### Step 2: Select Your Database

In the left panel:

**Servers**

└─ **PostgreSQL**

└─ **Databases**

└─ **company\_db**

**Click on your database (e.g., company\_db).**

---

### Step 3: Open Query Tool

- Right-click on your database
- Click Query Tool

A new SQL editor window will open.

---

### Step 4: Write the CREATE TABLE Query

```
CREATE TABLE employee (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    age INT,  
    salary NUMERIC  
);
```

---

### Step 5: Run the Query

Click the ► Execute button (or press F5).

If successful, you'll see:

Query returned successfully.

---

### Step 6: Verify the Table

In the left panel:

```
Databases  
└─ company_db  
    └─ Schemas  
        └─ public  
            └─ Tables
```

Right-click Tables → Refresh

You should see:

employee

# ALTER Command (DDL)

The ALTER command is used to modify the structure of an existing table.

You can:

- Add a column
- Modify a column
- Drop a column
- Rename a column

---

## 1. ADD COLUMN

Syntax

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example

```
ALTER TABLE Students  
ADD Email VARCHAR(100);
```

Before

StudentID Name Age

After

StudentID Name Age Email

---

## 2. MODIFY COLUMN

Syntax (MySQL)

```
ALTER TABLE table_name  
MODIFY column_name new_datatype;
```

Example

```
ALTER TABLE Students  
MODIFY Name VARCHAR(100);
```

Changes the maximum length of Name from 50 to 100 characters.

---

### 3. DROP COLUMN

#### Syntax

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

#### Example

```
ALTER TABLE Students  
DROP COLUMN Email;
```

Removes the Email column.

---

### 4. RENAME COLUMN

#### Syntax (MySQL)

```
ALTER TABLE table_name  
RENAME COLUMN old_name TO new_name;
```

#### Example

```
ALTER TABLE Students  
RENAME COLUMN Name TO StudentName;
```

Changes column name from Name to StudentName.

---

#### Complete Example

##### Create Table

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

##### Add Column

```
ALTER TABLE Students  
ADD Email VARCHAR(100);
```

##### Modify Column

**ALTER TABLE Students  
MODIFY Name VARCHAR(100);**

**Rename Column**

**ALTER TABLE Students  
RENAME COLUMN Name TO StudentName;**

**Drop Column**

**ALTER TABLE Students  
DROP COLUMN Email;**

---

### Quick Summary

| ALTER Operation | Example                                                 |
|-----------------|---------------------------------------------------------|
| Add column      | ALTER TABLE Students ADD Email VARCHAR(100);            |
| Modify column   | ALTER TABLE Students MODIFY Name VARCHAR(100);          |
| Drop column     | ALTER TABLE Students DROP COLUMN Email;                 |
| Rename column   | ALTER TABLE Students RENAME COLUMN Name TO StudentName; |

## DROP Command (DDL)

The DROP command is used to permanently delete a database object such as a database, table, view, or index.

 **Warning:** Once dropped, the object and its data are removed permanently.

---

### 1. DROP DATABASE

**Syntax**

**DROP DATABASE database\_name;**

**Example**

**DROP DATABASE CollegeDB;**

**Result: The entire CollegeDB database is deleted.**

---

## **2. DROP TABLE**

**Syntax**

**DROP TABLE table\_name;**

**Example**

**DROP TABLE Students;**

**Result: The Students table and all its records are deleted.**

---

## **3. DROP VIEW**

**Syntax**

**DROP VIEW view\_name;**

**Example**

**DROP VIEW StudentNames;**

**Result: The StudentNames view is deleted.**

---

## **4. DROP INDEX**

**Syntax (MySQL)**

**DROP INDEX index\_name  
ON table\_name;**

**Example**

**DROP INDEX idx\_name  
ON Students;**

**Result: The index idx\_name is removed.**

---

**Complete Example**

**CREATE DATABASE CollegeDB;**

**USE CollegeDB;**

**CREATE TABLE Students (**  
    **StudentID INT,**  
    **Name VARCHAR(50)**  
**);**

**DROP TABLE Students;**

**DROP DATABASE CollegeDB;**

---

### **DROP vs TRUNCATE**

| Command  | Deletes Data                            | Deletes Structure                       |
|----------|-----------------------------------------|-----------------------------------------|
| DROP     | <input checked="" type="checkbox"/> Yes | <input checked="" type="checkbox"/> Yes |
| TRUNCATE | <input checked="" type="checkbox"/> Yes | <input checked="" type="checkbox"/> No  |

#### **Example**

**DROP TABLE Students;**

 Table removed completely.

**TRUNCATE TABLE Students;**

 All rows removed, but the table still exists.

#### **Quick Memory Tip**

**DROP = Delete the object itself (table/database/view) permanently.** ☒

## **TRUNCATE Command (DDL)**

The **TRUNCATE** command is used to remove all rows (records) from a table while keeping the table structure intact.

---

### **Syntax**

**TRUNCATE TABLE table\_name;**

---

### **Example**

#### **Create Table**

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

#### **Insert Data**

```
INSERT INTO Students VALUES (1, 'Amit', 20);  
INSERT INTO Students VALUES (2, 'Neha', 21);
```

#### **Before TRUNCATE**

| StudentID | Name | Age |
|-----------|------|-----|
| 1         | Amit | 20  |
| 2         | Neha | 21  |

#### **Execute TRUNCATE**

```
TRUNCATE TABLE Students;
```

#### **After TRUNCATE**

**StudentID Name Age**

**Result: All records are deleted, but the Students table still exists.**

---

#### **Verify Table Still Exists**

```
SELECT * FROM Students;
```

#### **Output:**

**Empty set**

**The table exists, but it contains no rows.**

---

## TRUNCATE vs DELETE

| Feature                      | TRUNCATE                            | DELETE                              |
|------------------------------|-------------------------------------|-------------------------------------|
| Removes all rows             | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| Can use WHERE                | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| Keeps table structure        | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| Faster for deleting all rows | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |

### Example

Delete all rows:

```
TRUNCATE TABLE Students;
```

Delete specific rows:

```
DELETE FROM Students  
WHERE Age < 18;
```

---

### Quick Memory Tip

TRUNCATE = Empty the table, but keep the table. ☒

- DROP → Deletes the table itself.
- TRUNCATE → Deletes all data only.
- DELETE → Deletes selected rows (or all rows if no WHERE clause is used).

## RENAME Command (DDL)

The RENAME command is used to change the name of a database object, most commonly a table.

---

### 1. Rename a Table

## **Syntax (MySQL)**

**RENAME TABLE old\_table\_name TO new\_table\_name;**

### **Example**

**Suppose we have a table named Students.**

**RENAME TABLE Students TO StudentInfo;**

**Before**

**Students**

**After**

**StudentInfo**

**The table name changes, but all data remains intact.**

---

## **Complete Example**

### **Create Table**

**CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);**

### **Rename Table**

**RENAME TABLE Students TO StudentInfo;**

### **Check Table**

**SELECT \* FROM StudentInfo;**

**Now you must use StudentInfo instead of Students.**

---

## **Rename a Column (using ALTER)**

**Many databases rename columns through ALTER TABLE.**

### **Syntax**

**ALTER TABLE table\_name**

**RENAME COLUMN old\_column\_name TO new\_column\_name;**

## Example

```
ALTER TABLE StudentInfo  
RENAME COLUMN Name TO StudentName;
```

---

## Before and After

### Before

StudentID Name Age

### After

|           |             |     |
|-----------|-------------|-----|
| StudentID | StudentName | Age |
|-----------|-------------|-----|

---

## Quick Summary

|               |                                                            |
|---------------|------------------------------------------------------------|
| Task          | Command                                                    |
| Rename table  | RENAME TABLE Students TO StudentInfo;                      |
| Rename column | ALTER TABLE StudentInfo RENAME COLUMN Name TO StudentName; |

## Memory Tip

RENAME = Change the name, keep the data and structure unchanged. ☒

## COMMENT Command (DDL)

The COMMENT command is used to add descriptive notes (comments) to database objects such as tables and columns.

Comments help explain the purpose of tables or columns but do not affect the data.

---

### 1. Comment on a Table

#### Syntax (MySQL)

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype  
) COMMENT = 'table description';
```

**Example**

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
) COMMENT = 'Stores student information';
```

**Comment: "Stores student information"**

---

## **2. Comment on a Column**

**Syntax**

```
column_name datatype COMMENT 'description'
```

**Example**

```
CREATE TABLE Students (  
    StudentID INT COMMENT 'Unique student ID',  
    Name VARCHAR(50) COMMENT 'Student full name',  
    Age INT COMMENT 'Student age'  
);
```

---

## **3. Add Comment Using ALTER**

**Syntax**

```
ALTER TABLE table_name  
MODIFY column_name datatype  
COMMENT 'description';
```

**Example**

```
ALTER TABLE Students  
MODIFY Name VARCHAR(50)  
COMMENT 'Student complete name';
```

---

**Complete Example**

```
CREATE TABLE Students (  
  StudentID INT COMMENT 'Unique ID',  
  Name VARCHAR(50) COMMENT 'Student Name',  
  Age INT COMMENT 'Student Age'  
) COMMENT = 'Stores student details';
```

---

### **Purpose of COMMENT**

- Documents the database structure.
- Makes tables and columns easier to understand.
- Useful for developers and database administrators.

### **Example Meaning**

**Name VARCHAR(50) COMMENT 'Student full name'**

This tells anyone reading the schema that the Name column stores the student's full name.

---

### **Quick Memory Tip**

**COMMENT = Add a note/description to a table or column for documentation purposes. ☒**

### **DDL Commands Summary:**

- **CREATE** → Create objects
- **ALTER** → Modify objects
- **DROP** → Delete objects
- **TRUNCATE** → Remove all rows
- **RENAME** → Change object name
- **COMMENT** → Add descriptions/notes to objects

# Interview Question and answer

## CREATE Command

1. What is the CREATE command?

Answer: CREATE is a DDL command used to create database objects such as databases, tables, views, and indexes.

2. What is the syntax of CREATE DATABASE?

CREATE DATABASE CollegeDB;

3. What is the difference between CREATE DATABASE and CREATE TABLE?

- CREATE DATABASE creates a new database.
- CREATE TABLE creates a table inside a database.

4. What is a VIEW?

Answer: A view is a virtual table based on the result of a SELECT query.

5. Why do we use INDEX?

Answer: An index improves the speed of data retrieval.

---

## ALTER Command

6. What is ALTER used for?

Answer: ALTER modifies the structure of an existing table.

7. What operations can be performed using ALTER?

- Add column
- Modify column
- Drop column
- Rename column

8. How do you add a column?

ALTER TABLE Students

ADD Email VARCHAR(100);

9. How do you rename a column?

**ALTER TABLE Students  
RENAME COLUMN Name TO StudentName;**

**10. How do you remove a column?**

**ALTER TABLE Students  
DROP COLUMN Email;**

---

#### **DROP Command**

**11. What is DROP?**

**Answer: DROP permanently deletes a database object and its data.**

**12. How do you drop a table?**

**DROP TABLE Students;**

**13. How do you drop a database?**

**DROP DATABASE CollegeDB;**

**14. Can dropped data be recovered?**

**Answer: Generally no, unless a backup exists.**

---

#### **TRUNCATE Command**

**15. What is TRUNCATE?**

**Answer: TRUNCATE removes all rows from a table but keeps the table structure.**

**16. Syntax of TRUNCATE?**

**TRUNCATE TABLE Students;**

**17. Difference between DELETE and TRUNCATE?**

| <b>DELETE</b>                | <b>TRUNCATE</b>         |
|------------------------------|-------------------------|
| <b>Removes selected rows</b> | <b>Removes all rows</b> |
| <b>Can use WHERE</b>         | <b>Cannot use WHERE</b> |
| <b>Slower</b>                | <b>Faster</b>           |
| <b>DML</b>                   | <b>DDL</b>              |

**18. Does TRUNCATE remove the table?**

**Answer: No, it only removes the data.**

---

### **RENAME Command**

**19. What is RENAME used for?**

**Answer: It changes the name of a database object.**

**20. How do you rename a table?**

**RENAME TABLE Students TO StudentInfo;**

**21. Does renaming affect data?**

**Answer: No, only the object's name changes.**

---

### **COMMENT Command**

**22. Why is COMMENT used?**

**Answer: To add descriptions to tables and columns.**

**23. Does COMMENT affect data?**

**Answer: No, it is used only for documentation.**

**24. Example of column comment?**

**Name VARCHAR(50) COMMENT 'Student Full Name'**

---

### **Comparison Questions**

**25. Difference between DROP and TRUNCATE?**

**DROP**

**TRUNCATE**

**Deletes table + data**

**Deletes only data**

**Structure removed**

**Structure remains**

**Cannot use table again Table remains usable**

**26. Difference between DELETE and DROP?**

**DELETE**

**DROP**

**Removes rows Removes object**

**DELETE          DROP**

**Table remains   Table removed**

**DML              DDL**

**27. Difference between CREATE and ALTER?**

**CREATE              ALTER**

**Creates new object   Modifies existing object**

---

### **Frequently Asked Viva Questions**

**28. What does VARCHAR(50) mean?**

**Answer: Variable-length text with a maximum of 50 characters.**

**29. What is a Primary Key?**

**Answer: A column that uniquely identifies each row in a table.**

**30. What is NOT NULL?**

**Answer: Prevents a column from storing NULL values.**

**31. What is CHECK Constraint?**

**Answer: Ensures data satisfies a condition.**

**Example:**

**Age INT CHECK (Age >= 18)**

**32. What is an Index?**

**Answer: A database object that speeds up searching and retrieval.**

**33. What are DDL commands?**

**Answer: Commands that define or modify database structures.**

**DDL Commands:**

**CREATE**

**ALTER**

**DROP**

**TRUNCATE**

**RENAME**

**COMMENT**

### **Top 5 Interview Questions**

- 1. What is DDL and name its commands?**
- 2. Difference between DELETE, DROP, and TRUNCATE?**
- 3. What is ALTER TABLE used for?**
- 4. What is the purpose of an INDEX?**
- 5. What is a VIEW and why is it used?**